

PocketSmart AI — AI-Assisted Goal-Based Budget Planner

Project Description

PocketSmart AI is a web-based planning assistant built around a simple idea: **start with the spending limit, then plan the purchase**. A user chooses a planning area—home setup, event preparation or jewelry—and enters a total budget with a few requirements. Gemini produces a structured recommendation, while the FastAPI backend independently checks the calculations before the result is displayed.

The system is not a payment platform and does not claim to know live retailer prices. Its purpose is to reduce planning effort and make the budget decision visible.

Main Scenarios

Scenario 1: Home Setup

A user enters a total budget and quantities for lights, fans, furniture and dining tables, along with the rooms they are working on. The system creates a category-wise plan and provides search links for further shopping.

Scenario 2: Event Preparation

A user enters the event type, guest count, venue preference and total budget. The planner distributes the amount across venue, catering, decoration and entertainment, while retaining a contingency where appropriate.

Scenario 3: Occasion Jewelry

A user enters an occasion, budget and style preferences. An optional outfit photo can be supplied so the multimodal AI can consider visible colours and style when generating suggestions.

Technical Architecture

The application uses a three-part structure:

- 1. **Frontend:** Jinja2 templates, HTML, CSS and JavaScript.
- 2. **Backend:** FastAPI, Pydantic validation, authentication, budget verification and SQLite storage.
- 3. **External AI:** Google Gemini for structured recommendations and optional image understanding.

Project Workflow

Stage	Activity	Main Files
1	Environment and AI configuration	.env.example, config.py, check_gemini.py
2	Input and authentication	models.py, auth.py, main.py
3	AI recommendation generation	gemini_utils.py
4	Budget verification and shopping links	gemini_utils.py
5	User interface	templates/, static/
6	Storage and history	database.py
7	Testing	tests/ and 6.Project Testing/evidence/

Engineering Workflow

The most important implementation rule is:

AI suggestion → backend sanitisation → budget calculation → final plan.

This prevents the generated response from becoming the application’s source of truth for arithmetic.

Conclusion

PocketSmart AI demonstrates how a generative model can be used as a planning component inside a conventional web application. The project combines AI-generated recommendations with deterministic application rules so that users receive a useful plan while retaining visibility and control over their spending limit.